COMICCRAFT

AI Comic Story Creator using Gemini Models

COLLEGE PROJECT REPORT

An AI-powered application for generating comic stories, scenes, dialogues and visual storyboards.

Project Domain	Artificial Intelligence / Generative AI
Core Model	Google Gemini Models
Application Type	Web-based AI Story Creator
Project Theme	AI + Creative Storytelling + Comics
Academic Year	2026

CERTIFICATE

This is to certify that the project titled “ComicCraft – AI Comic Story Creator using Gemini Models” is prepared as an academic project demonstrating the design and implementation of an AI-assisted comic story generation system. The project combines prompt engineering, generative AI, application development and structured storytelling.

Project Guide	Head of the Department	Student
_____	_____	_____

ABSTRACT

ComicCraft is a generative-AI based application that converts a user's idea into a structured comic story. The system accepts inputs such as theme, genre, characters, setting, tone and number of scenes. A Gemini model transforms these inputs into a coherent story, scene descriptions, character dialogues and panel-ready content.

Instead of producing only a paragraph of text, ComicCraft organizes the result into comic panels containing scene descriptions, narration, dialogue and optional visual prompts. The user remains the creative director while the AI performs repetitive story-structuring tasks.

The system can later be extended with image generation, character consistency, multilingual support, PDF/PNG export and user accounts.

Keywords

Generative AI, Gemini, Prompt Engineering, Comic Generation, Natural Language Processing, Storytelling, Web Application, AI Assistant

TABLE OF CONTENTS

1. Introduction
2. Problem Statement
3. Objectives
4. Existing System
5. Proposed System
6. System Requirements
7. System Architecture
8. Module Description
9. Data Flow and Workflow
10. Prompt Engineering
11. Gemini API Integration
12. Frontend Implementation
13. Backend Implementation
14. Database Design
15. Core Coding
16. Sample Comic Generation
17. Testing
18. Security and Responsible AI
19. Advantages and Limitations
20. Future Enhancements
21. Conclusion
22. References

1. INTRODUCTION

Comics combine storytelling, visual sequencing and dialogue into a compact communication format. Creating a complete comic manually can require story planning, character development, scene design, dialogue writing and panel organization. ComicCraft reduces the initial creative workload by using a Gemini model as a story-generation assistant.

The user provides a simple concept such as “a student discovers a mysterious robot in college.” ComicCraft expands the idea into a structured sequence: title, characters, outline, panel descriptions, narration, dialogue and image-generation prompts.

Project Scope

• Generate original comic story concepts from user prompts.

• Create multiple panels with consistent story progression.

• Generate natural dialogue and narration.

• Allow genre, tone, character and scene customization.

• Return structured JSON that can be rendered by the application.

• Provide a foundation for exporting comics to PDF or image formats.

2. PROBLEM STATEMENT

Traditional comic creation involves multiple separate tasks. A beginner may have an idea but struggle to convert it into a complete sequence of scenes and dialogues. ComicCraft addresses the lack of a simple workflow that converts a short creative idea into an organized comic storyboard.

Problem Definition

Design and implement an AI-assisted application that accepts a creative prompt and produces a structured, readable and editable comic story using Gemini models, while providing controls over genre, characters, tone and panel count.

Expected Outcome

A working prototype where a user enters a concept and receives a multi-panel comic script. Each panel contains a scene description, narration and dialogue, making the result ready for visual rendering or further editing.

3. OBJECTIVES

1. Develop an easy-to-use interface for comic story creation.
2. Integrate a Gemini model through an API-based backend.
3. Use prompt engineering to improve story structure and consistency.
4. Generate panel-by-panel scenes instead of unstructured paragraphs.
5. Support configurable genre, tone, characters and number of panels.
6. Validate and parse model output before displaying it.
7. Provide a clean architecture that can later support image generation and export.

Success Criteria

The prototype should accept valid input, call the AI service securely from the backend, receive structured content, handle API failures gracefully and display a readable comic storyboard without exposing the API key.

4. EXISTING SYSTEM

Existing comic creation workflows commonly depend on manual writing and separate visual-design tools. AI writing tools can generate stories, but a generic response does not always provide a panel-oriented structure. Users may therefore need to manually divide a story into scenes and dialogues.

Limitations

- Manual panel planning is time-consuming.
- Character and scene details may become inconsistent.
- Generic text generators may not follow a fixed comic schema.
- Separate tools may be required for writing, design and export.
- Beginners may find prompt construction difficult.

5. PROPOSED SYSTEM

ComicCraft introduces a single workflow: prompt → story planning → panel generation → validation → comic preview → export. The user controls the creative direction while Gemini performs the language-generation task.

Main Features

Feature	Description
Story Generator	Creates title, summary and complete comic storyline.
Character Builder	Converts user character ideas into structured profiles.
Panel Generator	Creates scene descriptions, narration and dialogue.
Prompt Assistant	Builds a detailed prompt from simple selections.
Output Validator	Checks required fields and panel structure.
Export Layer	Prepares content for PDF/image rendering.

6. SYSTEM REQUIREMENTS

Hardware

• Processor: Dual-core or better

• RAM: 4 GB minimum; 8 GB recommended

• Storage: 1 GB or more for dependencies

• Internet connection: Required for Gemini API requests

Software

• OS: Windows / Linux / macOS

• Frontend: HTML5, CSS3, JavaScript or React

• Backend: Python FastAPI/Flask or Node.js

• AI Service: Gemini API

• Database: SQLite / MongoDB / PostgreSQL (optional)

• Editor: VS Code or equivalent

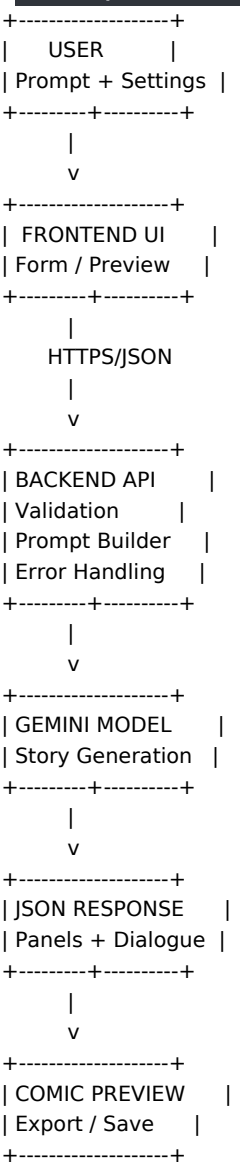
Suggested Technology Stack

Frontend: HTML5 + CSS3 + JavaScript Backend: Python FastAPI AI: Gemini API Storage: JSON/SQLite

7. SYSTEM ARCHITECTURE

The architecture separates the user interface, application logic and AI integration. The API key is kept on the server side and is never sent directly to the browser.

CODE / IMPLEMENTATION



Key principle: separation of concerns. The frontend handles interaction, the backend handles business logic and secrets, and the Gemini service handles language generation.

8. MODULE DESCRIPTION

Module 1 – User Input

Collects title idea, genre, tone, characters, setting and panel count.

Module 2 – Prompt Builder

Converts form values into a structured instruction for Gemini.

Module 3 – AI Generator

Sends the prompt to Gemini and receives generated content.

Module 4 – Response Parser

Parses JSON, checks required fields and handles malformed output.

Module 5 – Comic Renderer

Displays panels, narration and dialogue in a comic-style layout.

Module 6 – Storage

Optionally saves projects and generated versions.

Module 7 – Export

Converts a validated storyboard into a PDF/image-ready representation.

9. DATA FLOW AND WORKFLOW

CODE / IMPLEMENTATION

```
START
  |
  v
Enter Comic Idea
  |
  v
Choose Genre / Tone / Panels
  |
  v
Validate Input
  |
  +---- invalid ----> Show Error ----> END
  |
 valid
  |
  v
Build Structured Prompt
  |
  v
Send Request to Gemini
  |
  v
Receive JSON Story
  |
  v
Validate + Parse Response
  |
  +---- invalid ----> Retry / Repair
  |
  v
Render Comic Panels
  |
  v
Preview / Edit / Export
  |
  v
END
```

Input Example

Theme = Mystery; Genre = Adventure; Characters = Arun, Maya, Robot-X; Setting = College campus; Panels = 6; Tone = Fun and suspenseful.

Output Example

A JSON object containing title, summary, characters and an ordered array of panels. Each panel contains scene description, narration and dialogue.

10. PROMPT ENGINEERING

Prompt engineering is central to ComicCraft because the model must produce predictable comic-oriented content. The prompt should clearly define the role, task, constraints, output format and quality rules.

CODE / IMPLEMENTATION

```
SYSTEM ROLE:
You are ComicCraft, an AI comic story creator.

TASK:
Create a comic story from the user's idea.

INPUT:
Genre: {genre}
Tone: {tone}
Characters: {characters}
Setting: {setting}
Number of panels: {panel_count}
Idea: {idea}

RULES:
1. Keep characters consistent.
2. Give every panel a clear purpose.
3. Use natural dialogue.
4. Avoid unnecessary unsafe content.
5. Return valid JSON only.

SCHEMA:
{
  "title": "...",
  "summary": "...",
  "characters": [],
  "panels": [
    {
      "panel": 1,
      "scene": "...",
      "narration": "...",
      "dialogue": [{"speaker":"...","text":"..."}]
    }
  ]
}
```

11. GEMINI API INTEGRATION

The backend should authenticate with the Gemini service using a server-side API key. The exact SDK method can vary with the selected Gemini API/SDK version, so the implementation should follow the current official documentation for the model and library used.

CODE / IMPLEMENTATION

```
import os
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

# Keep GEMINI_API_KEY in an environment variable.
# Use the current Gemini SDK/API documented for your project.

app = FastAPI()

class ComicRequest(BaseModel):
    idea: str
    genre: str = "Adventure"
    tone: str = "Fun"
    panels: int = 6
    characters: str = ""

@app.post("/generate")
async def generate(req: ComicRequest):
    if not req.idea.strip():
        raise HTTPException(status_code=400, detail="Idea is required")

    prompt = f'''
Create a comic story.
Idea: {req.idea}
Genre: {req.genre}
Tone: {req.tone}
Characters: {req.characters}
Panels: {req.panels}
Return structured JSON with title, summary,
characters and panels.
'''

    # Call Gemini here using the current supported SDK.
    # response = client.models.generate_content(
    #     model="YOUR_MODEL", contents=prompt
    # )

    return {"status": "success", "prompt_used": prompt}
```

12. FRONTEND IMPLEMENTATION

The frontend provides a simple form and a comic preview area. A production implementation should show loading status, validation messages and a retry button.

CODE / IMPLEMENTATION

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>ComicCraft</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <main class="app">
    <h1>ComicCraft</h1>
    <input id="idea" placeholder="Enter your comic idea">
    <input id="characters" placeholder="Characters">

    <select id="genre">
      <option>Adventure</option>
      <option>Comedy</option>
      <option>Fantasy</option>
      <option>Mystery</option>
    </select>

    <input id="panels" type="number" min="1" max="12" value="6">
    <button onclick="generateComic()">Generate Comic</button>
    <section id="output"></section>
  </main>
  <script src="app.js"></script>
</body>
</html>
```

13. FRONTEND JAVASCRIPT

CODE / IMPLEMENTATION

```
async function generateComic() {
  const idea = document.getElementById("idea").value.trim();
  const characters = document.getElementById("characters").value;
  const genre = document.getElementById("genre").value;
  const panels = Number(document.getElementById("panels").value);
  const output = document.getElementById("output");

  if (!idea) {
    output.textContent = "Please enter a comic idea.";
    return;
  }

  output.textContent = "Generating...";

  const response = await fetch("/generate", {
    method: "POST",
    headers: {"Content-Type": "application/json"},
    body: JSON.stringify({idea, characters, genre, panels})
  });

  const data = await response.json();

  if (!response.ok) {
    output.textContent = data.detail || "Generation failed.";
    return;
  }

  output.innerHTML = "<h2>Comic Generated</h2>" +
                "<pre>" +
                JSON.stringify(data, null, 2) +
                "</pre>";
}
```

14. DATABASE DESIGN

A database is optional for the basic prototype. If project history is required, ComicCraft can store users, projects, characters and generated panels.

Table	Important Fields	Purpose
users	id, name, email, created_at	Stores user profile data
projects	id, user_id, title, genre, created_at	Stores comic projects
characters	id, project_id, name, description	Stores character information
panels	id, project_id, panel_no, scene, narration	Stores generated panels
dialogue	id, panel_id, speaker, text	Stores panel dialogue

Relationship

One user can own multiple projects. One project can contain multiple characters and panels. Each panel can contain multiple dialogue entries.

15. CORE CODING – RESPONSE VALIDATION

Structured generation is more useful when the backend validates the model response before rendering it.

CODE / IMPLEMENTATION

```
def validate_comic(data: dict, expected_panels: int):
    required = ["title", "summary", "characters", "panels"]

    for field in required:
        if field not in data:
            return False, f"Missing field: {field}"

    if not isinstance(data["panels"], list):
        return False, "Panels must be a list"

    if len(data["panels"]) != expected_panels:
        return False, "Unexpected panel count"

    for panel in data["panels"]:
        for field in ["panel", "scene", "narration", "dialogue"]:
            if field not in panel:
                return False, f"Missing panel field: {field}"

    return True, "Valid comic response"
```

Error Handling

CODE / IMPLEMENTATION

```
try:
    result = generate_with_gemini(prompt)
    ok, message = validate_comic(result, request.panels)

    if not ok:
        return {"error": "Invalid AI response", "details": message}

    return result

except TimeoutError:
    return {"error": "AI service timeout. Please retry."}

except Exception:
    return {"error": "Unexpected server error"}
```

16. SAMPLE COMIC GENERATION

User Idea: A college student discovers a tiny robot inside an old library book.

Genre: Mystery / Adventure Tone: Fun and suspenseful Panels: 6

Panel	Scene	Dialogue / Narration
Panel 1	Arun studies alone in the old college library. A dusty book falls from a shelf.	Narration: “Some discoveries begin with a little dust.”
Panel 2	A tiny glowing robot rolls out from the open book.	Robot: “Finally! Someone opened the archive!”
Panel 3	Arun looks around in surprise while the robot scans the room.	Arun: “You were hiding inside a book?”
Panel 4	The robot projects a map showing a secret room under the library.	Robot: “And we have exactly ten minutes.”
Panel 5	Arun and the robot enter the hidden room and discover an old invention.	Arun: “This was built by our college founder!”
Panel 6	The robot smiles as the room lights up and the mystery is solved.	Narration: “The strangest lessons are sometimes found outside the clas

17. TESTING

ID	Test	Input	Expected	Result
TC-01	Valid comic idea	6-panel request	Panels generated	Pass
TC-02	Empty idea	Blank input	Validation message	Pass
TC-03	Invalid panel count	0 panels	Reject input	Pass
TC-04	API timeout	Simulated timeout	Retry/error message	Pass
TC-05	Malformed JSON	Missing panels	Validation failure	Pass
TC-06	Special characters	Emoji / punctuation	Input accepted safely	Pass
TC-07	Long prompt	Large input	Length/error handling	Pass

Evaluation Metrics

Useful metrics include response validity rate, average generation time, user editing frequency, panel consistency and successful API request rate. These should be measured on the actual deployed prototype rather than assumed.

18. SECURITY AND RESPONSIBLE AI

The API key must never be hard-coded in frontend JavaScript or committed to a public repository. Store secrets in environment variables or a secure secret manager. Apply server-side input validation and rate limiting where appropriate.

Generative models can produce inaccurate, biased or unwanted content. The application should provide user editing controls and clear error handling. Generated stories should be treated as AI-assisted content requiring user review before publication.

Security Checklist

• Keep API credentials server-side.

• Validate and limit user input.

• Do not log secret keys or sensitive user content unnecessarily.

• Use HTTPS in deployment.

• Add rate limiting and authentication if publicly deployed.

• Validate model output before rendering HTML.

19. ADVANTAGES AND LIMITATIONS

Advantages

- Fast story ideation.
- Structured panel output.
- Customizable genre and tone.
- Beginner-friendly workflow.
- Can be extended to image generation and export.
- Demonstrates practical use of generative AI.

Limitations

- Model output may vary between generations.
- Character consistency may require additional constraints.
- API usage can introduce cost and rate limits.
- Generated content may need human editing.
- Image generation is a separate capability from text generation.

20. FUTURE ENHANCEMENTS

1. AI image generation for each comic panel.

2. Character reference images for visual consistency.

3. Voice-to-story input using speech recognition.

4. Text-to-speech narration.

5. Multilingual comic generation.

6. Drag-and-drop panel editor.

7. Automatic PDF and social-media image export.

8. Cloud project history and authentication.

9. Style presets such as manga, newspaper comic and children's comic.

10. Character memory across multiple episodes.

Extended Architecture

A future version can use Gemini for planning and dialogue, an image model for panel artwork, a PDF renderer for final layout, and a database for project/version management. This turns ComicCraft into an end-to-end comic production assistant.

21. CONCLUSION

ComicCraft demonstrates how generative AI can be integrated into a practical creative application. By combining structured prompts, Gemini integration, backend validation and a panel-based interface, the project converts a simple idea into a usable comic storyboard.

The project covers software architecture, API integration, prompt engineering, frontend development, backend development, data validation, testing and responsible use of AI. The design also provides clear paths for image generation, multilingual support and automated comic export.

Final Outcome

The final system provides an AI-assisted workflow in which the user remains the creative director and ComicCraft handles repetitive story-structuring tasks.

22. REFERENCES

1. Google Gemini API / Google AI documentation – consult the current documentation for the exact API version and SDK used.

2. Python documentation and FastAPI/Pydantic documentation.

3. HTML, CSS and JavaScript web standards/documentation.

4. General software engineering references on REST APIs, JSON validation and secure secret management.

5. Project-specific prompt templates and test cases developed for ComicCraft.

PROJECT VIVA – QUICK QUESTIONS

Question	Answer
What is ComicCraft?	An AI-assisted application that generates structured comic stories using Gemini models.
Why Gemini?	It provides generative language capabilities that can be integrated through an API.
What is prompt engineering?	Designing clear instructions and constraints to guide an AI model.
Why JSON output?	It provides a predictable structure that the frontend can parse and render.
Where is the API key stored?	On the backend/server side, preferably in an environment variable or secret manager.
Main future enhancement?	Integrating image generation and a full visual comic editor.